

ROBIS SIMULATION

Deployment and User Guide for Gazebo, Cartographer Mapping, Map Export, and A* + TEB Navigation

WORKSPACE	Portable ROS Noetic catkin workspace
TESTED ON	Ubuntu 20.04.6 LTS, ROS Noetic, ARM64
SIMULATION	Gazebo 11.15.1
SLAM	Cartographer ROS 1.0.0
NAVIGATION	move_base, A* global planner, TEB local planner

This guide describes the clean, distributable workspace. Generated build files, tuning backups, runtime logs, and navigation recording tools are not part of the release.

Release guide - 20 September 2026

1. Workspace Overview

RobiS Simulation provides a complete ROS 1 workflow for a four-wheel mobile robot. The workspace can be uploaded to a Git repository, cloned onto a new Ubuntu 20.04 computer, rebuilt locally, and used without fixed absolute paths.

- Robot model: rectangular chassis, four wheels, lidar riser, and cylindrical 2D lidar.
- Simulation world: 10 m x 10 m floor, boundary walls, four box obstacles, and one cylinder.
- Mapping: Cartographer 2D SLAM with RViz visualization.
- Map output: Cartographer `.pbstream`, occupancy-grid `.pgm`, and portable `.yaml` metadata.
- Navigation: Cartographer localization, `move_base`, A* global planning, and TEB local control.
- Remote-friendly defaults: Gazebo server enabled, Gazebo graphical client disabled, RViz enabled.

Robot dimensions

Item	Dimensions / placement
Chassis	423.5 mm long x 268 mm wide x 66.78 mm high
Wheels	126 mm diameter x 41 mm wide
Wheel centers	+/-152 mm longitudinal, +/-154 mm lateral from chassis center
Lidar	102 mm diameter x 85 mm high; center 135 mm forward
Lidar riser	Approximately 174 mm high between chassis and lidar

2. Tested Environment

The following versions were read directly from the remote computer used for functional verification. The project uses standard ROS Noetic APIs and should also build on x86_64 Ubuntu 20.04 when the same dependencies are installed.

Component	Verified version
Operating system	Ubuntu 20.04.6 LTS (Focal Fossa)
CPU architecture	ARM64 / aarch64
ROS distribution	ROS Noetic
roslaunch	1.17.0
Gazebo	11.15.1
Cartographer ROS	1.0.0, built from source
TEB local planner	0.9.1
Navigation stack	1.17.3

The tested machine loads Cartographer from `/home/robotcar/cartographer`. That location is not hard-coded. The environment helper discovers common locations or accepts an explicit `CARTOGRAPHER_WS` or `CARTOGRAPHER_SETUP`.

3. Required Software

Base platform

- Ubuntu 20.04 LTS.
- ROS Noetic Desktop Full, which supplies ROS core tools and common desktop components.
- A catkin build environment and `rosdep`.
- Cartographer and Cartographer ROS, installed through apt when available or built from source.

Install project dependencies

```
sudo apt update
sudo apt install -y \
  python3-rosdep \
  ros-noetic-xacro \
  ros-noetic-robot-state-publisher \
  ros-noetic-joint-state-publisher \
  ros-noetic-gazebo-ros \
  ros-noetic-gazebo-plugins \
  ros-noetic-rviz \
  ros-noetic-map-server \
  ros-noetic-move-base \
  ros-noetic-global-planner \
  ros-noetic-teb-local-planner \
  ros-noetic-teleop-twist-keyboard
```

Install Cartographer from packages

```
sudo apt install -y ros-noetic-cartographer ros-noetic-cartographer-ros
```

If these packages are unavailable from the configured apt source, build Cartographer and Cartographer ROS from source in a separate workspace. Then set one of the following before sourcing the RobiS helper:

```
export CARTOGRAPHER_WS=/path/to/cartographer_ws
# or
export CARTOGRAPHER_SETUP=/path/to/cartographer_ws/devel_isolated/setup.bash
```

4. Deploy and Build

The release root is already a catkin workspace and contains `src/CMakeLists.txt`. Clone it to the desired workspace path, install dependencies, and build locally.

```
git clone https://github.com/Robify-Robotics/Robify-Robotics-RobiS.git ~/robis_sim_ws
cd ~/robis_sim_ws
source /opt/ros/noetic/setup.bash
rosdep update
rosdep install --from-paths src --ignore-src -r -y
catkin_make
```

Source this helper in every new terminal:

```
source ~/robis_sim_ws/src/robis_simulation/scripts/robis_sim_env.bash
```

Do not upload or copy `build/`, `devel/`, `install/`, runtime logs, ROS bags, CSV files, Python caches, or tuning backups. They are machine-specific and are excluded by the root `.gitignore`.

5. Default Startup and Gazebo GUI

Launch behavior	Mapping	Navigation
Gazebo server (`gzserver`)	Started	Started
Gazebo graphical client (`gzclient`)	Not started	Not started
RViz	Started	Started

The default starts the physics simulation and sensors without opening the Gazebo window. This reduces graphical load while preserving robot motion, lidar, mapping, localization, and navigation.

PERFORMANCE NOTE: Gazebo is computationally expensive, especially when its graphical client and RViz run together on a remote or lower-power computer. Visible stutter, delayed screen refresh, or slower-than-real-time simulation can be normal. A graphical pause does not automatically mean that TEB or Cartographer has failed.

Open Gazebo during mapping

```
roslaunch robis_simulation cartographer_mapping.launch gazebo_gui:=true
```

Open Gazebo during navigation

```
roslaunch robis_simulation navigation_cartographer_teb.launch gazebo_gui:=true
```

Start Gazebo only

```
# Graphical client enabled by default for this standalone launch.
roslaunch robis_simulation gazebo.launch
```

```
# Headless alternative.
roslaunch robis_simulation gazebo.launch gui:=false
```

For a remote desktop, make sure the active desktop owns the configured `DISPLAY` and `XAUTHORITY`. Mapping and navigation launch files accept both as arguments.

6. Create a Map

Terminal 1: start mapping

```
source ~/robis_sim_ws/src/robis_simulation/scripts/robis_sim_env.bash
roslaunch robis_simulation cartographer_mapping.launch
```

Terminal 2: drive the robot

```
source ~/robis_sim_ws/src/robis_simulation/scripts/robis_sim_env.bash
roslaunch teleop_twist_keyboard teleop_twist_keyboard.py cmd_vel:=/cmd_vel
```

- Use moderate speed and avoid sustained contact with walls or obstacles.
- Traverse each accessible corridor and scan obstacle surfaces from several angles.
- Return near the starting position to help Cartographer close the loop.
- Keep the mapping launch running until the map has been saved.

7. Save the Map

Run the following in a third terminal while mapping is still active. The commands finish trajectory 0, save Cartographer state, export a 0.05 m occupancy grid, and make the YAML image path portable.

```
source ~/robis_sim_ws/src/robis_simulation/scripts/robis_sim_env.bash

rosservice call /finish_trajectory "trajectory_id: 0"

rosservice call /write_state "filename: '${ROBIS_SIM_WS}/src/robis_simulation/maps/robis_map.pbstream'
include_unfinished_submaps: true"

roslaunch cartographer_ros cartographer_pbstream_to_ros_map \
  -map_filestem=${ROBIS_SIM_WS}/src/robis_simulation/maps/robis_map \
  -pbstream_filename=${ROBIS_SIM_WS}/src/robis_simulation/maps/robis_map.pbstream \
  -resolution=0.05

sed -i 's|^image:.*|image: robis_map.pgm|' \
  "${ROBIS_SIM_WS}/src/robis_simulation/maps/robis_map.yaml"
```

Expected output files are `robis\_map.pbstream`, `robis\_map.pgm`, and `robis\_map.yaml` in the package `maps` directory. Wait for all commands to finish, then stop the mapping launch with `Ctrl-C`.

8. Start Navigation

```
source ~/robis_sim_ws/src/robis_simulation/scripts/robis_sim_env.bash
roslaunch robis_simulation navigation_cartographer_teb.launch
```

Use `2D Nav Goal` in RViz to select a target position and heading. The runtime uses the saved occupancy map and `pbstream` state. Cartographer maintains `map -> odom`; `global\_planner/GlobalPlanner` supplies the A* path; `teb\_local\_planner/TebLocalPlannerROS` produces local velocity commands.

Useful launch overrides

```
# Disable RViz.
roslaunch robis_simulation navigation_cartographer_teb.launch start_rviz:=false

# Use a different map and Cartographer state.
roslaunch robis_simulation navigation_cartographer_teb.launch \
  map_file:=/absolute/path/map.yaml \
  pbstream_file:=/absolute/path/map.pbstream
```

9. Validation and Troubleshooting

Structural validation

```
python3 ~/robis_sim_ws/src/robis_simulation/test/validate_model.py
```

Useful runtime checks

```
roscall node list
rostopic info /scan
rostopic echo -n 1 /map --noarr
roslaunch tf tf_echo map base_link
rosparam get /move_base/base_global_planner
rosparam get /move_base/base_local_planner
```

Symptom	Check
RViz is red immediately after launch	Wait for Gazebo to spawn the robot and publish `/scan`; TF may take a few seconds to connect.
No map appears	Confirm `/scan` has `/gazebo` as publisher and `/cartographer_node` as subscriber.
Navigation does not start	Confirm all three saved map files exist and source the environment helper again.
Motion or display stutters	Close the Gazebo client, keep `gzserver`, and compare ROS timestamps with wall time.
Cartographer package not found	Set `CARTOGRAPHER_WS` or `CARTOGRAPHER_SETUP` before sourcing the helper.

10. Release Checklist

- Repository root contains ``src/CMakeLists.txt`` and ``src/robis_simulation``.
- No ``build``, ``devel``, ``install``, runtime-log, recording, cache, or tuning-backup files are included.
- ``maps/robis_map.yaml`` uses the relative image path ``robis_map.pgm``.
- The target computer installs ROS Noetic, Gazebo ROS plugins, navigation packages, TEB, teleoperation, and Cartographer.
- The workspace is rebuilt on the target computer with ``catkin_make``.
- The environment helper is sourced before mapping, saving, or navigation.
- Mapping and navigation are tested separately, and each launch is stopped cleanly with ``Ctrl-C``.

Verified release behavior: the workspace builds successfully; mapping publishes an occupancy grid and a connected ``map -> base_link`` transform; Cartographer exposes ``finish_trajectory`` and ``write_state``; navigation loads the A* global planner and TEB local planner; default mapping and navigation do not start the Gazebo graphical client.